VERIFICATION PLAN
APB System Verification

Document Information	
Document ID	[APB-2026-001] https://github.com/tnl3pdx/ece593-apb-verification
Project / Product	APB System Verification
Design Version	RTL v1.1
Verification Level	Sub-System
Prepared By	Truong Le, Marco Martinez Corral (ECE 593 Spring 2026 Team 2)
Reviewed By	N/A
Approved By	N/A
Date Created	[2026-15-04]
Last Revised	[2026-30-04]
Revision	0.3 - Draft
Status	[X] Draft [ ] In Review [ ] Approved [ ] Superseded
Confidentiality	[X] Public [ ] Internal [ ] Confidential [ ] Restricted

Purpose Statement

This Verification Plan defines the strategy, environment, tools, and completion criteria for the functional verification of APB System Verification. It is structured in accordance with the hierarchical verification methodology described in Comprehensive Functional Verification by Bruce Wilde.

Table of Contents

Table of Contents 2
1. Introduction 2
1.1 Purpose and Scope 3
1.2 Design Overview 3
1.3 Verification Hierarchy 3
1.4 References 3
1.5 Definitions, Acronyms, and Abbreviations 3
2. Functions to Be Verified 4
3. Functions Not Covered at This Verification Level 5
4. Verification Strategy and Methods 5
4.1 Overall Strategy 5
4.2 Testbench Architecture 5
4.2.1 Testbench Components 5
4.3 Stimulus Methods 6
4.3.1 Directed Tests 6
4.3.2 Constrained-Random Tests 6
4.3.3 Assertion-Based Verification 6
4.3.4 Formal Verification 6
4.4 Checking Strategy 6
5. Verification Environment 7
5.1 Simulation Environment 7
5.2 Coverage Plan 7
6. Required Tools 7
7. Completion Criteria 8
7.1 Simulation Completion Criteria 8
7.2 Formal Verification Completion Criteria 9
7.3 Sign-Off Checklist 9
8. Resources and Schedule 9
8.1 Human Resources 9
8.2 Hardware Resources 10
8.3 Schedule 10
9. Risk and Mitigation 10
10. Document Revision History 11

1. Introduction

1.1 Purpose and Scope

This Verification Plan (VP) describes the complete approach for functionally verifying APB System Verification. It establishes the verification scope, methods, environment, required tools, completion criteria, and resource plan.

This document applies to the following verification level:

- [ ] Block-level verification
- [x] Sub-system verification
- [ ] Full-chip / Top-level verification
- [ ] System-level verification

1.2 Design Overview

The design under verification is an APB system consisting of 3 components: a controller and 2 peripherals. The function of this system is to establish the use of APB to communicate data (read/write) between the controller and a particular peripheral.

The design under verification (DUV) is a custom AMBA Advanced Peripheral Bus (APB) sub-system. It consists of 1 APB Master controller and 2 APB peripherals (Slaves): a standard 32-bit memory array (Slave 0 at base address `0x0000_0000`) and a custom Timer module (Slave 1 at base address `0x0000_0080`). The protocol operates with a 32-bit data bus and a 32-bit address bus. The system's primary function is to route read/write commands from the master to the targeted peripheral using the APB non-pipelined, two-phase (Setup and Access) protocol, properly handling wait states (`pready`).

1.3 Verification Hierarchy

Verification of this design follows a hierarchical approach consistent with Bruce Wilde's Comprehensive Functional Verification framework. Each level of the hierarchy addresses a specific set of functions and feeds results to the next level.

Hierarchy Level	Design Scope	Covered In This VP	Referenced VP
Block	APB Master and Slaves (Memory/Timer)	[ ] Yes [X] No	[APB-MAS-2026-001], [APB-TP-2026-001], [APB-MP-2026-001]
Sub-system	APB System	[X] Yes [ ] No	[APB-2026-001]
Full-chip	N/A	[ ] Yes [x] No	N/A
System	N/A	[ ] Yes [x] No	N/A

1.4 References

Ref #	Document Title	Version	Date
[R01]	Design Specification – APB System Verification	[1.0]	[2026-04-19]
[R02]	Comprehensive Functional Verification – Bruce Wilde	1st edition	2005
[R03]	AMBA APB Protocol Specification	[Issue E]	[2023-02-28]
[R04]	Class Testbench Architecture Report	[1.0]	[2026-04-30]
[R05]	Exclusion List	[1.0]	[2026-04-30]

1.5 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
DUV	Design Under Verification
DUT	Device Under Test
UVM	Universal Verification Methodology
SV	SystemVerilog
TB	Testbench
FC	Functional Coverage
CC	Code Coverage
CR	Coverage Report
VP	Verification Plan
APB	Advanced Peripheral Bus

2. Functions to Be Verified

This section lists all functions and features of the DUV that are within scope of this verification plan, as required by the verification hierarchy (Wilde, Chapters 3–4). Each function is assigned a unique identifier for traceability.

FV-ID	Function / Feature Description	Design Ref	Priority	Verification Method
FV-001	Reset behavior – all registers reset to defined values	DS §2.1	Critical	Directed + Functional Coverage
FV-002	Correct setup for read/write cycles	DS §3.2	Critical	Directed + Constrained-Random
FV-003	Interface protocol compliance – APB3	DS §4.0	High	Protocol VIP + Constrained-Random
FV-004	Data integrity – read/write correctness across all modes	DS §5.1	High	Scoreboard-based checking
FV-005	Interrupt handling – priority, masking, and clearing	DS §6.0	High	Directed + Constrained-Random
FV-006	Timer peripheral function - set/start/check status	DS §7.0	High	Directed Sequences
FV-007	Error Detection: Transaction Errors	DS §8.0	High	Fault injection
FV-009	Register map – read/write/reset value for all registers	DS §10	Critical	Register Model (RAL) Automation

FV-ID	Function / Feature Description	Design Ref	Priority	Verification Method
FV-010	Read/Write with Waitstate	DS §11	High	Directed Sequences

Priority legend: Critical = must pass before tape-out; High = required for signoff; Medium = best-effort; Low = stretch goal.

3. Functions Not Covered at This Verification Level

The following functions are explicitly excluded from this plan because they will be verified at a different level of the verification hierarchy. A corresponding Verification Plan reference is provided for each item.

NC-ID	Function / Feature	Rationale for Exclusion	Verified In (VP Reference)
NC-001	AHB-to-APB bridge timing and arbitration	Requires full-chip context	[System VP – VP-SYS-001]
NC-002	Multi-master arbitration and bus contention	Single-master configuration chosen; multi-master scenarios deferred	[Analog VP – VP-AMS-001]
NC-003	Software driver interaction	Covered at system / emulation level	[System VP – VP-SYS-002]
NC-004	Formal exhaustive proof of FSM state reachability	Out of project scope (Milestones 1 - 5)	N/A - Not planned

4. Verification Strategy and Methods

Milestone 1 utilizes a traditional flat SystemVerilog testbench for basic interconnect routing sanity checks. Subsequent milestones will migrate to a UVM environment utilizing constrained-random stimulus, scoreboard-based checking, and functional coverage to achieve signoff (Milestone 5 and final deliverables).

4.1 Overall Strategy

The verification strategy follows a layered approach combining:

- Directed tests – to verify specific corner cases and compliance requirements.
- Constrained-random stimulus – to explore unanticipated scenarios and achieve broad coverage.
- Assertion-based verification (ABV) – to catch protocol violations and temporal properties at the point of occurrence.
- Functional coverage – to measure completeness of the constrained-random campaign.
- Code coverage – to identify unreached HDL constructs.

4.2 Testbench Architecture

Milestone 1 uses a traditional (non-class-based) flat SystemVerilog testbench. The top-level module APB_TB instantiates APB_SYS_DUT, drives clock and reset, and applies directed stimulus via task-based helper routines (resetDUT, doTransact). No UVM or interface objects are used at this milestone. Subsequent milestones will migrate to a class-based (MS2) and then UVM 1.2 (MS4) environment.

4.2.1 Testbench Components

Component	Description	Reuse / New
Top-level testbench	Instantiates DUV, connects clocks/resets and interfaces	New
UVM Test	Controls test scenario; extends uvm_test	New
UVM Env	Assembles agents, scoreboards, and coverage collectors	New
Agent(s)	One per interface; contains driver, monitor, sequencer	[ ] Reused [x] New
Scoreboard	Compares DUV output against reference model	[ ] Reused [x] New
Reference Model	Golden behavioral model of DUV	[ ] Reused [x] New
Coverage Collector	Implements functional coverage groups and bins	New
Assertion Library	SVA properties bound to DUV interfaces and internals	[ ] Reused [x] New
Register Model (RAL)	UVM register model generated from IP-XACT / RALF	[ ] Generated [x] Manual

4.3 Stimulus Methods

4.3.1 Directed Tests

Milestone 1: Directed tests exercise fundamental APB read/write paths using the flat testbench task doTransact. Each test drives PADDR, PWDATA, and the R/W select signal, then checks that PRDATA and transfer-status outputs match expected values.

Directed testing will be used to check for various cases in the design. This testing will include the simple read/write test to different peripherals. For corner cases, this will check the boundary addresses of the memory peripheral and check the accessibility of registers in the timer peripheral.

- Reset and initialization sequences
- Sequential Reads/Writes
- Read/Write with Waitstates
- Error injection sequences (e.g., illegal transactions)
- Timer Setup/Start/Checking

4.3.2 Constrained-Random Tests

Constrained-random stimulus is planned from Milestone 2 onward. In MS2, a class-based Generator will produce randomized APBTransaction objects constrained to legal address ranges and data values. Coverage feedback (MS3) will guide generation toward under-exercised bins.

- Random transaction ordering with protocol-legal constraints
- Randomized delay injection between transactions
- Weighted coverage-driven generation for under-covered bins
- Address randomization constrained to valid Slave 0 / Slave 1 address spaces

4.3.3 Assertion-Based Verification

Assertion-based verification is planned from Milestone 3 onward. Initial assertion targets are listed below; implementation will be added to the class-based TB.

- Interface protocol assertions (e.g., valid/ready handshake, request-grant timing)

- Data-path integrity assertions (e.g., no X propagation on outputs)
- Safety/liveness properties bound using bind constructs
- APB Bus idle assertion

4.3.4 Formal Verification

Formal property checking is not planned within the MS1–MS5 scope of this project. The entries below are retained for completeness per the template; they may be revisited for future courses.

- Control FSM state reachability and deadlock freedom
- FIFO empty/full flag correctness
- Reset completeness proof

4.4 Checking Strategy

Result checking will use the following methods:

- Scoreboard-based checking – DUV outputs are compared transaction-by-transaction against the reference model.
- In-line assertion checking – SVA monitors detect protocol violations immediately.
- End-of-simulation report – summary of all pass/fail checks, coverage, and assertion statistics.

5. Verification Environment

This section describes the simulation and verification environment, in accordance with Wilde Chapters 3, 4, and 7–12.

5.1 Simulation Environment

Simulation Environment Details	
Simulator	Synopsys VCS
Simulator Version	VCS 2023.03-SP2 (PSU ECE Linux Lab)
Verification Methodology	UVM 1.2
HDL Language	SystemVerilog IEEE 1800-2017
Assertion Language	SystemVerilog Assertions (SVA)
Compile/Sim Options	Compile: -sverilog -debug_access+all -kdb -cm line+cond+fsm+tgl+branch -ntb_opts uvm Sim: -l run.log -cm line+cond+fsm+tgl+branch
Seed Management	TBD
Regression Framework	Makefile

5.2 Coverage Plan

The following table outlines the functional coverage strategy for the AMBA APB system. To systematically track our progress toward verification closure, key architectural requirements have been mapped to specific SystemVerilog covergroups. This plan captures critical operational metrics, including baseline APB phase transitions and data integrity crosses and the directed edge-case sequences validating the custom timer peripheral. By measuring these specific bins using Synopsys VCS, we will quantify the completeness of the testbench and formally prove that all targeted functional scenarios have been successfully exercised.

FV-ID	Coverage Group Name	Bins / Cross Points	Target (%)	Tool
FV-001	cg_reset	All registers × reset state	100%	VCS
FV-002	cg_apb_operations	Check read/write, check address access, and cross of the two. Check read/write transitions.	90%	VCS
FV-003	cg_error_resp	All transaction types × error / no-error	95%	VCS
FV-004	cg_data_integrity	All address × preset data patterns × operation type	90%	VCS
FV-005	cg_timer_validation	Floor timer, access valid	100%	VCS

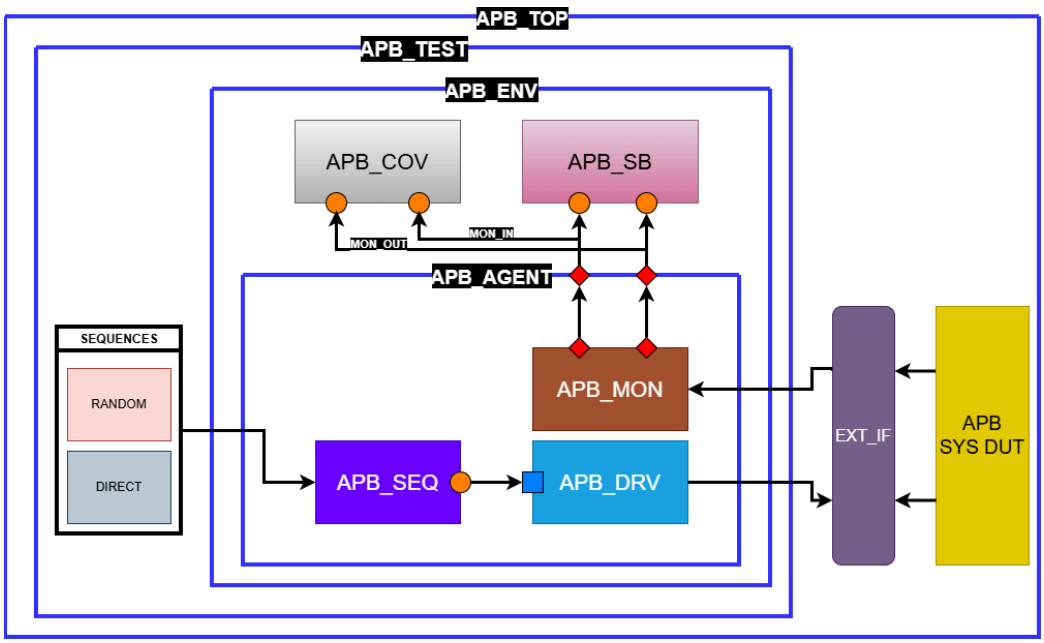
5.3 UVM Environment Architecture

The following table describes the components used in the APB UVM environment.

UVM Component	File Name	Description
Sequence Item	APB_TRANSACTION.sv	Each item holds data to drive the DUV via randomized addresses and data (constrained to legal peripherals). The address is created using the slave selection and register selection. Data is randomized with random values and with random preset values. For the protocol, a read/write direction is selected as well. The outputs that are logged with the transaction is the data out (applicable for only reads), the transfer status (the error signal), a valid bit (signaled only for reads), and a timestamp for logging. There is an additional illegal bit that is used to signify that the transaction is intentionally illegal. This is so that the error checking of the DUV can be exercised, and that the golden models are not touched by these transactions.
Sequence	APB_SEQUENCE.sv	This file contains multiple sequences that are used to test with the DUV. There is a standard sequence called rand_seq for the constrained-random testing. In addition, there are several direct sequences used to test specific behavior of the DUV.
Sequencer	APB_SEQUENCER.sv	This component simply takes the sequence in the environment and passes it to the driver.
Driver	APB_DRIVER.sv	The driver stimulates the DUV's data, address, and control signals using the sequence data provided by the sequencer.
Monitor	APB_MONITOR.sv	The monitor is split into two parts: monitor_input and monitor_output. Each task observes the virtual interface and samples the interface depending on the control signals that indicate a request or response. The two tasks will write the transaction data into separate uvm_analysis_port instances.
Scoreboard	APB_SCOREBOARD.sv	The scoreboard provides both a golden model for the memory modules and the timer module in the design. The two main tasks that are forked are the get_input and get_output. Each task gets transactions from the monitor via a uvm_analysis_export interface that connects to a uvm_tlm_analysis_fifo. The input task will take the transactions and apply the same stimulus to the golden models. When the output tasks receive the output transaction, a write will check if the correct control signals were observed, and a read will obtain the data from the golden models and compare the transactions to see if they match. There is also a way to check for illegal transactions. These transactions are not written into the golden models, but will

UVM Component	File Name	Description
		check if the environment correctly observes that the DUT returned an error or not.
Coverage	APB_COVERAGE.sv	The coverage is implemented as a component in the UVM environment. The environment connects the monitor's two analysis ports to the coverage component, so that the covergroups can sample the incoming transactions from the monitor.
Agent	APB_AGENT.sv	The agent instantiates the monitor, driver, and sequencer as one combined object.
Environment	APB_ENV.sv	The environment instantiates the agent and scoreboard. It also connects the monitor's uvm_analysis_port instances to the scoreboard and coverage's uvm_analysis_imp instances.
Test	APB_TEST.sv	The test instantiates the environment and runs the test sequences. This also prints the topology of the UVM architecture.
Top	APB_TOP.sv	This is a module that instantiates the DUT, the interface, clock/reset generation, registers the interface with the UVM DB, and runs the tests. The interface is connected from the DUT to the UVM environment via a virtual interface.

Below is a figure depicting the current UVM architecture. Sequences of both random and direct types are sent to the sequencer, which then relays them to the driver. The driver stimulates the DUV, and the monitor then samples the interface's inputs and outputs. Through two analysis ports on the monitor: one from inputs to the DUV and one for output from the DUV, the scoreboard can check the inputs with its golden models, and the coverage can sample the inputs and outputs for its covergroups.



The next figure shows the hierarchical structure of the UVM environment. The structure follows the same layering as the diagram.

```
Name                      Type
-------------------------------------------------------
uvm_test_top              apb_test
  env                     apb_env
    agnt                  apb_agent
      drv                 apb_driver
        rsp_port          uvm_analysis_port
        seq_item_port     uvm_seq_item_pull_port
      mon                 apb_monitor
        ap_in             uvm_analysis_port
        ap_out            uvm_analysis_port
      seqr                apb_sequencer
        rsp_export        uvm_analysis_export
        seq_item_export   uvm_seq_item_pull_imp
        arbitration_queue array
        lock_queue        array
        num_last_reqs     integral
        num_last_rsps     integral
    cov                   apb_coverage
      cov_mon_in_port     uvm_analysis_imp_mon_in
      cov_mon_out_port    uvm_analysis_imp_mon_out
    scb                   apb_scoreboard
      scb_mon_in_port     uvm_analysis_imp_mon_in
      scb_mon_out_port    uvm_analysis_imp_mon_out
```

For connecting the monitor to the scoreboard and coverage components, two uvm_analysis_port instances are created on the monitor to send over input and output transaction data on separate connections. The scoreboard and coverage components connect to these analysis ports using uvm_analysis_imp_mon_in and uvm_analysis_imp_mon_out. Multiple analysis implementations can be made using this macro. A "_mon_in", and "_mon_out" analysis implementation was created in the APB_PARAMS.sv file.

```
`uvm_analysis_imp_decl(_mon_in)
`uvm_analysis_imp_decl(_mon_out)
```

Both the scoreboard and coverage components instantiate these implementations, and the environment connects the monitor's analysis ports to the implementations.

```
uvm_analysis_imp_mon_in #(apb_transaction, apb_scoreboard) scb_mon_in_port;
uvm_analysis_imp_mon_out #(apb_transaction, apb_scoreboard) scb_mon_out_port;
```

```
uvm_analysis_imp_mon_in  #(apb_transaction, apb_coverage) cov_mon_in_port;
uvm_analysis_imp_mon_out #(apb_transaction, apb_coverage) cov_mon_out_port;
```

```
// Connect to Scoreboard
agnt.mon.ap_in.connect(scb.scb_mon_in_port);
agnt.mon.ap_out.connect(scb.scb_mon_out_port);

// Connect to Coverage (Both Interfaces)
agnt.mon.ap_in.connect(cov.cov_mon_in_port);
agnt.mon.ap_out.connect(cov.cov_mon_out_port);
```

6. Required Tools

This section lists all software tools required to support the described verification environment, consistent with Wilde Chapters 5–6. Items marked [Procure] may drive the software procurement team; items marked [Internal] require support from internal EDA/flow teams.

Category	Tool / Package	Version	Vendor / Source	Status
Simulator	VCS	2023.12-SP2	Synopsys	[X] Licensed [ ] Procure
Formal Tool	N/A	[X.X]	N/A	[ ] Licensed [ ] Procure
Coverage Merge	VCS Coverage	[X.X]	Synopsys	[X] Licensed [ ] Procure
VIP	N/A	[X.X]	N/A	[ ] Licensed [ ] Procure
Regression	[LSF / Slurm / Make-based]	[X.X]	[Internal EDA Flow Team]	[ ] Available [ ] Internal
Linting	[Spyglass / Questa Lint]	[X.X]	[Synopsys / Siemens]	[ ] Licensed [ ] Procure
Clock Domain Check	[Spyglass CDC / 0-In CDC]	[X.X]	[Synopsys / Mentor]	[ ] Licensed [ ] Procure
Version Control	Git	N/A	GitHub	[X] Available [ ] Internal
Bug Tracking	Git	N/A	GitHub	[X] Available [ ] Internal
Documentation	Google Docs	N/A	Google	[X] Available [ ] Internal

7. Completion Criteria

Verification is declared complete when ALL of the following measurable criteria have been satisfied, in accordance with Wilde Chapter 13. Each criterion must be formally signed off by the Verification Lead and Design Owner.

7.1 Simulation Completion Criteria

#	Criterion	Target Threshold	Achieved Value
1	All directed tests pass with zero unexpected failures	100% pass rate	[100] / [%]
2	Constrained-random regression passes across N seeds using a class-based testbench	N ≥ 50 seeds (per Milestone 2)	[ ] / [N]
3	All functional coverage groups close	≥ [95]%	[ ] / [%]
4	Statement and branch code coverage achieved	≥ [90]%	[ ] / [%]

#	Criterion	Target Threshold	Achieved Value
5	Toggle coverage achieved on DUV outputs and interfaces	≥ [85]%	[ ] / [%]
6	FSM state and transition coverage achieved	100% reachable states	[100] / [%]
7	All SVA assertions pass (zero assertion failures in final regression)	0 failures	[ ] / [N]
8	All open bugs triaged; no Critical or High severity bugs outstanding	0 Critical / High open	[ ] / [N]
9	Waived coverage items documented and approved	Approval on record	[X] Done

7.2 Formal Verification Completion Criteria (TBD)

#	Criterion	Target	Status
1	All formal properties proven or vacuity-free	PROVEN / BOUNDED	[ ] Done
2	No unreachable properties (vacuity checks passed)	0 vacuous passes	[ ] Done
3	All counterexamples analyzed and resolved	0 open CEX	[ ] Done

7.3 Sign-Off Checklist

Sign-Off Item	Responsible Party	Date	Sign
Simulation completion criteria met (Section 7.1)	Verification Lead	4/30	MC
Formal completion criteria met (Section 7.2)	Verification Lead	N/A	N/A
Coverage waivers approved	Design Owner	4/30	TL
All high-priority bugs closed or deferred	Design Owner	4/30	TL
Verification plan reviewed and up-to-date	Verification Lead	[Date]	___
Regression results archived	Verification Lead	[Date]	___
Final sign-off approved by Program Management	Program Manager	[Date]	___

8. Resources and Schedule

This section defines the human, hardware, and software resources required, and provides schedule estimates to tie the verification plan to program management, as required by Wilde's framework.

8.1 Human Resources

Role	Name / Team	Allocation (FTE)	Responsibilities
Verification Lead	Truong/Marco	[X.X]	Plan, coordination, sign-off, risk management

Role	Name / Team	Allocation (FTE)	Responsibilities
Verification Engineer(s)	Truong/Marco	[X.X]	Testbench development, test writing, debug
Design Owner / RTL Engineer	Truong	[X.X]	Design clarifications, bug disposition
EDA / Flow Support	Marco	[X.X]	Tool setup, regression infrastructure
Program Manager	Marco	[X.X]	Schedule tracking, resource allocation, status reporting

8.2 Hardware Resources

Resource	Specification	Quantity	Availability
Simulation Compute Servers	PSU ECE Linux Servers	N/A	[X] Available [ ] Required
LSF / Job Scheduler Capacity	N/A	N/A	[x] Available [ ] Required
Formal Compute Servers	PSU ECE Linux Servers	N/A	[x] Available [ ] Required
Shared File Storage	PSU MCECS Storage	[X] TB	[x] Available [ ] Required
Emulation Platform	N/A	N/A	[x] N/A [ ] Required

8.3 Schedule

#	Milestone / Deliverable	Planned Start	Planned End	Owner	Status
M1	Verification Plan approved, RTL sourced, flat testbench creation and basic design functionality tested	[04/06]	[04/20]	Truong/Marco	[x] Open
M2	Class based testbench infrastructure complete, including generator, drivers, and other components. 20-50 randomized bursts of data verified.	[4/20]	[4/30]	Truong/Marco	[ ] Open
M3	Finalize RTL, complete class-based verification, add code/function coverage reports, update verification plan	[4/20]	[4/30]	RTL-Eng	[ ] Open
M4	Develop UVM testbench, define UVM architecture with updates to VP	[4/30]	[05/14]	V-Eng	[ ] Open
M5	UVM Architecture, environment, and testbench completion. All testcases completed and reflected in coverage reports.	[05/14]	[05/28]	V-Eng	[ ] Open

#	Milestone / Deliverable	Planned Start	Planned End	Owner	Status
	Bug injection & verification scenarios implemented. Final deliverables compiled: documents, RTL, files, report, presentation.				

9. Risk and Mitigation

#	Risk Description	Likelihood	Impact	Mitigation Plan
R1	RTL instability delaying testbench development	High	High	Freeze RTL interface specification; develop TB against stable interface spec first.
R2	Coverage closure not achieved within schedule	Medium	High	Add constrained-random coverage-driven tests; increase compute resources; waiver process for unreachable bins.
R3	Tool license unavailability causing simulation bottleneck	Low	High	Licenses available through PSU; schedule off-peak batch regressions, use “backup” servers if required.
R4	Key verification engineer departure / availability risk	Medium	Medium	Cross-train second engineer; maintain up-to-date documentation.
R5	Formal verification timeout on complex properties	Medium	Medium	Apply bounded model checking; abstract sub-blocks; consult formal tool support.
R6	Custom APB timer: blocked slave integration	Medium	Medium	Simplify design of timer, reduce complexity from parameterization. Worse-case: replace with another memory peripheral

10. Document Revision History

Revision	Date	Author	Description of Change	Approved By
0.1	[2026-04-17]	[Le, Martinez Corral]	Initial draft for internal review	Venkatesh Patil
0.2	[2026-04-30]	[Le, Martinez Corral]	Updated class-based TB and covergroups	Venkatesh Patil
0.3	[2026-05-15]	[Le, Martinez Corral]	Added UVM documentation and minor updates	Venkatesh Patil
1.0	[YYYY-MM-DD]	[Name]	Approved and baselined	[Approver]
1.1	[YYYY-MM-DD]	[Name]	[Description of change, e.g., Added Section 9]	[Approver]

End of Verification Plan | [Project Name] | [Document ID]